SpecTer - Proje ve Görev Sunumu

İşletim Sistemleri Dersi | 1. Üye: ShellCore (Çekirdek Kabuk & Süreç Yönetimi) **Süre: ~2.5 - 3 Dakika**

🎙️ Bireysel Sunum Konuşma Metni (Bunu Aynen Okuyabilir / Anlatabilirsin)

1. GİRİŞ VE PROJE ÖZETİ (30 SANİYE)

"Merhaba Hocam, arkadaşlar. Ekip olarak geliştirdiğimiz **SpecTer** projesi; Windows işletim sistemi üzerinde kernel seviyesinde süreç yönetimi, kaynak kısıtlaması (Sandboxing), anlık metrik izleme ve modern bir grafiksel arayüz sunan 4 modüllü hibrit bir terminal uygulamasıdır. Projemiz; kabuk motoru, Job Objects kısıtlayıcısı, donanım profilleyicisi ve TUI/GUI panelinin entegre çalışmasıyla oluşmaktadır."

2. BANA DÜŞEN KISIM: SHELLCORE MODÜLÜ (60 SANİYE)

"Ben bu projede sistemin omurgası ve kalbi olan **ShellCore** kısmını geliştirdim. ShellCore, sistemde iki kritik görevi yerine getiriyor:
Birincisi; Kullanıcıdan gelen komutları alıp işleyen REPL döngüsü ve tırnak duyarlı komut ayrıştırma motorudur (Tokenizer). Klasör ve dosya sistemi işlemlerini, süreç sonlandırmayı (`kill`), süreç listelemeyi (`ps`) ve donanım özetini (`sysinfo`) doğrudan yerel Win32 API fonksiyonlarıyla dahili komut olarak kabuğun kendi hafıza alanında yürüttüm.
İkincisi; Harici sistem uygulamalarını (örneğin `ping`, `gcc`, `cmd`) alt süreç (Child Process) olarak başlatan ve yöneten Win32 yürütme motorudur."

3. TEKNİK ZORLUKLAR VE KULLANDIĞIM İŞLETİM SİSTEMİ MEKANİZMALARI (60 SANİYE)

"İşletim sistemleri dersi açısından bu modülde 3 kritik mimari karar aldım:
1. **Doğrudan Win32 CreateProcessA ve Handle Yönetimi:** Kolaya kaçıp `system()` kullanmadık; çünkü `system()` arkada kontrolsüz bir cmd açar ve sürecin Kernel Handle'ına ulaşamazsınız. Biz `CreateProcessA` ile doğrudan sürecin `hProcess` tanıtıcısını ve PID'sini elde ettik.
2. **Boru Hatları (Anonymous Pipes) ve Deadlock Koruması:** Alt sürecin çıktısını (STDOUT ve STDERR) yakalamak için isimsiz borular kurduk. Ebeveyn süreçteki yazma uçlarını hemen kapatarak borularda kilitlenmeyi (deadlock) önledik ve GUI'ye canlı metin akışı sağladık.
3. **CREATE_SUSPENDED ile Takım Entegrasyonu:** Süreçleri askıda başlatabilme kabiliyeti ekledim. Süreç belleğe yüklendiği an henüz tek bir komut bile koşturmadan, ürettiğim `ProcessInfo` yapısını JobManager'a (2. üyeye) teslim ediyoruz. O RAM/CPU sınırını uyguladıktan sonra süreci `ResumeThread` ile güvenle başlatıyoruz."

4. KAPANIŞ VE SÖZÜ DEVRETME (15 SANİYE)

"Özetle ShellCore; komutun girilmesinden sürecin ayağa kaldırılmasına, çıktıların yakalanmasından diğer modüllerin tetiklenmesine kadar tüm sistemin veri sağlayıcısıdır. Şimdi oluşturduğum bu süreç tanıtıcısının Job Objects ile nasıl sınırlandırıldığını anlatması için sözü 2. üyemiz olan JobManager sorumlusuna bırakıyorum."

🎯 Hocadan Gelebilecek Zor Sorular ve Net Cevapların (Cheat Sheet)

Soru 1: 'cd' komutunu neden harici bir CreateProcessA süreci olarak çalıştırmadınız?

Cevap: Bir işletim sisteminde harici her süreç ebeveynin ortamını miras alır ama kendi izole bellek alanında çalışır. Eğer `cd` harici bir süreç olsaydı, alt sürecin çalışma dizini değişir ama bittiği an ana kabuğumuzun (Shell) dizini olduğu gibi

kalırdı. Bu sebeple kabuk dizinini değiştirmek için Win32 `SetCurrentDirectoryA` fonksiyonunu kabuğun kendi sürecinde dahili olarak çağırdık.

Soru 2: Anonymous Pipe (Boru Hattı) kurarken neden ebeveyndeki yazma ucunu kapattınız?

Cevap: Windows'ta bir borudan okuma yapan fonksiyon (`ReadFile`), boruya yazabilecek tüm tanıtıcılar (Write Handles) kapanana kadar dosya sonu (EOF) sinyali alamaz. Ebeveyn süreç kendi kopyasını kapatmazsa, alt süreç kapansa bile boru açık kalır ve ebeveyn sonsuza kadar okumaya çalışarak kilitlenir (Deadlock). Bu yüzden ebeveyn yazma ucunu hemen kapattık.

Soru 3: Süreci neden CREATE_SUSPENDED bayrağıyla başlattınız?

Cevap: Yarış durumunu (Race Condition) engellemek için. Süreci doğrudan başlatırsak, 2. üyemiz (JobManager) RAM sınırını koyana kadar alt süreç kötü niyetli veya aşırı bellek tüketen bir kod koşturup sistemi kilitleyebilir. Askıya alarak önce limitleri koyduk, ardından `ResumeThread` ile güvenle devam ettirdik.

📂 Kod Referans Tablosu (Kod Gösterirken Bakacağın Yerler)

Fonksiyon / Yapı	Satır Aralığı	İşlevi ve Kullanılan Win32 API
ProcessInfo struct	ShellCore.h : 15 - 24	Kernel Handle (`hProcess`), PID, TID ve Pipe çıktılarını tutan ana veri taşıyıcı.
parseCommand()	ShellCore.cpp : 109 - 137	Tırnak ve boşluk duyarlı durum makinesi (Tokenizer).
executeBuiltIn()	ShellCore.cpp : 159 - 548	`cd` , `dir` , `mkdir` , `del` , `sysinfo` , `ps` , `kill` yerel komutları.
launchProcess()	ShellCore.cpp : 550 - 665	`CreatePipe` , `CreateProcessA` , `ResumeThread` , Hook tetikleme.
executeLineStream()	ShellCore.cpp : 914 - 1090	GUI için `PeekNamedPipe` ile canlı çıktı akışı ve OEM->UTF-8 çevrimi.